CHAPTER 3

RESEARCH METHODOLOGY

This chapter is ordered so that each part is defined before it is used. Sections 3.1 to 3.3 describe the simulator, the robot and the vocabulary of reinforcement learning. Section 3.4 states the problem formally, Sections 3.5 to 3.7 build the task, the software and the safety cost that instantiate it, and Section 3.8 develops the algorithm that solves it. The remaining sections cover calibration and the training and evaluation protocols. The opening sections are deliberately descriptive: the argument for expressing a safety requirement as a constraint rather than as a reward term, and the published evidence behind the algorithm chosen, belong to Chapter 2 and are not restated here.

3.1 Simulation environment: Isaac Sim and Isaac Lab

NVIDIA Isaac Sim is a robotics simulator built on the Omniverse platform. Scenes are described in Universal Scene Description (USD) files, which carry a robot's link geometry, joint definitions, mass and inertia properties and collision meshes in one asset, and the physics is solved by PhysX. Rigid-body dynamics, contact resolution and articulation solving all execute on the GPU, so many independent copies of a scene advance together without their state returning to the CPU between steps. The asset that is simulated is also the asset that is rendered, which is how the gripper fault of Section 3.5 was diagnosed.

Isaac Lab is the reinforcement-learning framework layered on top. It is manager-based: an environment is assembled by declaring configuration objects for observations, actions, rewards, terminations, events and commands, rather than by writing a monolithic step function. A task defined this way is retargeted onto different hardware by replacing the robot articulation, the action term and the end-effector frame, leaving the reward and termination structure of a tested environment untouched. Isaac Lab has no publication of its own; its predecessor Orbit is the citable reference [**?**], and the GPU-resident training design both inherit was introduced with Isaac Gym [**?**].

Table 3.1 gives the software stack. Two of the versions are load-bearing rather than incidental: the RTX 5090 is a Blackwell GPU (compute capability sm_120), which is not supported by PyTorch builds before 2.7 or by NVIDIA drivers before the 570 series, so the CUDA 12.8 wheels and the 580-series driver are requirements.

The Isaac Lab version is held at exactly 2.3.0 rather than at the 2.3 release line. The line advanced to 2.3.1 during this work, and 2.3.1 requires a URDF importer version that the

Table 3.1. Software stack, as pinned for every run reported in this thesis.

Component	Version
Simulator	NVIDIA Isaac Sim 5.0.0
RL framework	Isaac Lab 2.3.0
On-policy trainer	`rsl_rl` 3.0.1 (installed by Isaac Lab)
Alternate RL library	`skrl` 2.1.0 (used only for the off-policy smoke test)
Python	3.11, conda environment `isaaclab`
Deep-learning framework	PyTorch 2.7.0+cu128
CUDA toolkit	12.8
GPU	NVIDIA RTX 5090, Blackwell, sm_120
NVIDIA driver	580 series
Host	Intel i9, 64 GB RAM, Ubuntu

installed Isaac Sim does not ship, which stopped training at start-up. Every run reported in Chapter 4 was produced from one frozen version of the code, with no change to the environment, the algorithm or the configuration between arms.

Key points

- Isaac Sim solves the physics on the GPU and Isaac Lab declares the task through managers, which is what makes thousands of parallel environments and a hardware retarget practical.
- The stack is pinned, not merely recorded. PyTorch 2.7 with CUDA 12.8 and a 580-series driver are requirements of the Blackwell GPU, not preferences.
- Isaac Lab is held at 2.3.0 exactly. The 2.3 line moved to 2.3.1 mid-project and broke start-up through a URDF importer version it requires.

3.2 The UR5e manipulator

The platform is a Universal Robots UR5e, a six-degree-of-freedom collaborative arm. Table 3.2 gives its published specifications [**?**] against the modelling decision each one drives, because several numbers appearing later in this chapter are datasheet limits carried into the simulator rather than values chosen for training convenience.

The last two rows carry a distinction that the rest of the chapter depends on. A real UR5e already has a certified safety controller. Nothing in this thesis replaces it, competes with it, or is evaluated against it. The simulation borrows the arm's physical envelope, its reach and its speed ceiling, and tests whether a learned policy can be held inside a kinematic safety budget by the optimiser itself, which is a different question from whether a hardware interlock can stop the arm.

Table 3.2. UR5e specifications and where each one enters the method.

Property	Value	Where it is used here
Degrees of freedom	6 revolute	Six-dimensional action space (Section 3.5.1)
Payload	5 kg	Not exercised; the grasp is a weld (Section 3.5)
Reach	850 mm	Bounds the goal-sampling box, far corner 0.84 m
Repeatability	$\pm$0.03 mm	Sets the scale at which a 1 cm goal tolerance is loose
Mass	20.6 kg	Not exercised; the arm is fixed-base in simulation
Maximum joint speed	180 deg/s (π rad/s)	Velocity limit in the simulated articulation
Safety functions	17, configurable	**Not used.** The learned constraint is what is under test
Certification	ISO 13849-1 Cat. 3 PL d; ISO 10218-1	Context only; no certified function is simulated

The six revolute joints are named in Table 3.3, with the working range of each and the home configuration the environment resets to. The joint-limit cost term of Section 3.7 measures clearance against these ranges, so the naming is used again there and in the per-joint reporting of Chapter 4.

Table 3.3. The six arm joints, their working range and the reset configuration used in this work.

Joint	Working range	Home (rad)	Home (deg)
`shoulder_pan`	$\pm$360°	0.00	0.0
`shoulder_lift`	$\pm$360°	-1.20	-68.8
`elbow`	$\pm$360°	1.40	80.2
`wrist_1`	$\pm$360°	-1.75	-100.3
`wrist_2`	$\pm$360°	-1.57	-90.0
`wrist_3`	$\pm$360°	0.00	0.0

The home pose places the arm reaching forward over the table with the wrist pointing down, which is the configuration every episode starts from before the cube and goal are randomised.

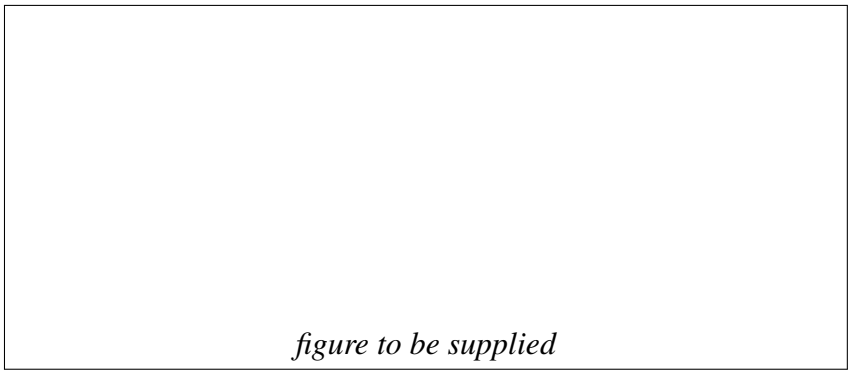

Figure 3.1. The UR5e in the simulated workspace.

3.3 Reinforcement learning, on-policy and off-policy

A reinforcement-learning agent observes the state of its environment, selects an action from a policy, receives a scalar reward, and lands in a new state. Repeating this produces a trajectory, and the object of training is a policy that maximises the reward accumulated along trajectories it generates. In this work the policy is a neural network that maps the observation vector of Section 3.5.1 to a distribution over six joint-position targets, and training adjusts the network weights.

Algorithms of this kind are organised by two questions: whether the agent has a model of the environment, and what it learns. Figure 3.2 shows the taxonomy. A model-based agent has, or learns, a function predicting state transitions and rewards, which lets it plan ahead, but a learned model is exploitable: an agent can find a policy that scores well against the model's errors and behaves badly in the real environment. This work is model-free, which is where the great majority of published manipulation results sit. Within model-free methods the split is between policy optimisation and Q-learning, with a group of algorithms interpolating between them, and that split lines up almost exactly with the on-policy and off-policy division described next.

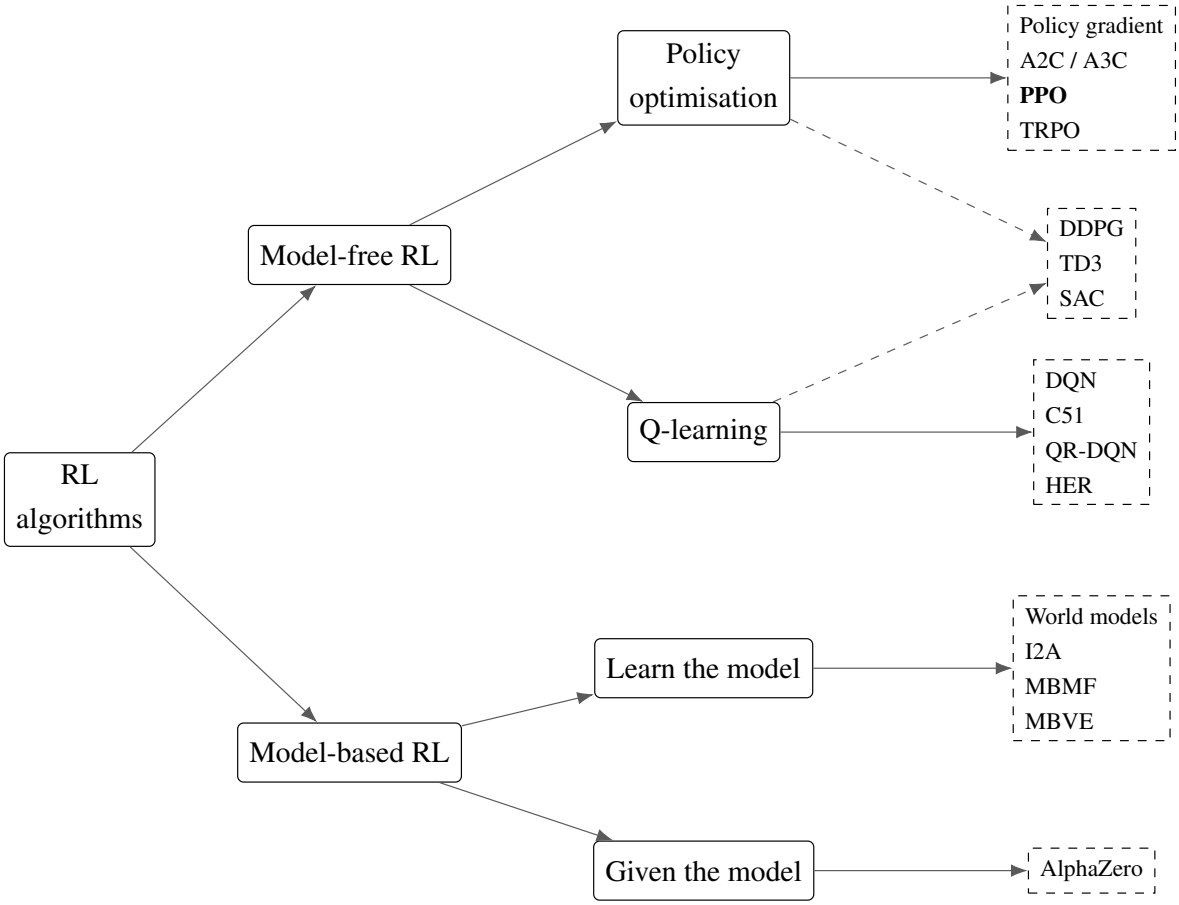

Figure 3.2. A taxonomy of reinforcement-learning algorithms. Redrawn from the taxonomy given in OpenAI's Spinning Up in Deep Reinforcement Learning [**?**]; the figure is an original rendering of that classification, not a reproduction of the original artwork. The algorithm used in this thesis is shown in bold.

Algorithms also divide by what data they may learn from. An *on-policy* method updates the policy using only transitions collected by the policy as it currently stands, discarding them once an update changes the policy. An *off-policy* method keeps a replay buffer and reuses transitions from earlier versions of the policy. Table 3.4 sets the two side by side.

Table 3.4. On-policy and off-policy methods compared.

Property	On-policy	Off-policy
Data used for an update	Only transitions from the current policy	Any past transition, from a replay buffer
Data reuse	Discarded after one update	Reused many times
Sample efficiency	Lower	Higher
Stability and reproducibility	Higher; optimises the distribution it visits	Lower; distribution shift between behaviour and target policy
Typical bottleneck	Gradient computation	Replay and value-estimation error
Suits	Cheap, massively parallel simulation	Expensive interaction, physical hardware
Example algorithms	**PPO**, TRPO, A2C / A3C, vanilla policy gradient	SAC, TD3, DDPG, DQN
Used in this thesis	Yes, PPO and its Lagrangian variant	No; retained as future work

This study is on-policy, and the reason is that its samples are cheap. At 4096 environments running in parallel on one GPU the bottleneck is the gradient computation, not the collection of experience, so the sample efficiency an off-policy method would buy has little to pay for itself with. Both families are established on robotic grasping [**?**], so this is a positioned choice rather than the only available one, and Chapter 6 returns to the off-policy comparison as the recognised complement to what is reported here. The particular on-policy algorithm used, and its constrained variant, are developed in Section 3.8, once the problem they solve has been stated.

Key points

- This work is model-free and on-policy, in the policy-optimisation branch of Figure 3.2.
- On-policy methods discard data after one update but stay stable and reproducible; off-policy methods reuse a replay buffer and are far more sample efficient.
- The choice here follows from cheap samples, not from off-policy methods being unsuitable. At 4096 parallel environments the gradient step is the bottleneck, not data collection.
- An off-policy comparison arm was pre-registered and cut on schedule grounds, so it is reported as a limitation rather than quietly omitted.

3.4 Problem formulation

The grasping task is posed as a constrained Markov decision process, or CMDP [**?**]. A standard Markov decision process is the tuple (S, A, P, r, γ) over a state space S, an action space A, a transition kernel P giving the probability of the next state under a chosen action, a reward function r and a discount factor γ. A CMDP augments that tuple with a cost function c, which scores each transition for hazard exactly as r scores it for progress, and a budget d. It asks for the policy that maximises expected discounted return subject to the expected accumulated cost staying within budget:

$$\max_{\pi} \mathbb{E}\left[\sum_{t} \gamma^{t} r(s_t, a_t)\right] \quad \text{subject to} \quad \mathbb{E}\left[\sum_{t} c(s_t, a_t)\right] \leq d. \tag{3.1}$$

The formulation separates *what the robot should achieve*, which is to lift the cube and bring it to a commanded pose, from *what it must not do*, which is to pass through configurations where the arm loses controllability or approaches its mechanical limits. The first is encoded in r and the second in c, and the two never mix. This is the structural property the whole comparison in Chapter 4 rests on: because no safety term appears in the reward, any safety difference measured between a constrained agent and an unconstrained one is attributable to the constraint machinery and not to a reward the two agents were optimising differently.

Three properties of Equation 3.1 shape how the results must be read, and stating them here avoids over-claiming later.

The constraint binds an **expectation, not a maximum**. A policy satisfying it is one whose *average* accumulated cost over episodes falls within d, which permits individual episodes to exceed the budget provided others compensate. A hardware safety case therefore has to be argued in terms of expected exposure over many operations, not in terms of a guaranteed worst case [**?**], and Chapter 4 reports a counter-result where the single worst episode is no better under the constraint than without it.

The cost is **undiscounted and episodic**. Where the return in Equation 3.1 is discounted by γ, the constrained quantity is a plain sum over a fixed 250-step episode. Hazard late in an episode therefore counts as heavily as hazard early in it, which is the correct treatment for a physical limit but ties the numerical value of d to the episode length. Changing the episode duration rescales the constraint and voids its calibration, a point Section 3.9 returns to.

The budget is **a single scalar over an aggregated cost**. Section 3.7 sums three separate hazard terms into one number before the constraint sees it, so one multiplier governs all three and the agent is free to trade between them. This keeps the optimisation simple at the price of not being able to state a separate guarantee per hazard, and the three terms are therefore reported separately even though they are constrained jointly.

Solving Equation 3.1 directly is awkward, since the constraint couples every timestep of every episode. The standard treatment, and the one used here, converts it into an unconstrained saddle-point problem through a Lagrange multiplier, which Section 3.8 develops. The remainder of the chapter fills in each element of the tuple in turn: the simulation environment and the MDP itself (Section 3.5), the software that implements them (Section 3.6), the safety cost that instantiates c (Section 3.7), the algorithm that solves the CMDP (Section 3.8), the calibration that makes the constraint meaningful rather than decorative (Section 3.9), and the training and evaluation protocol (Sections 3.10–3.11).

Key points

- The task is a CMDP: maximise expected return subject to expected accumulated cost staying within a budget d.
- Reward and cost are kept in separate channels, which is what makes the Chapter 4 comparison attributable to the constraint.
- The constraint binds an average, not a worst case, so the safety claim is about expected exposure rather than a guarantee.
- The cost is undiscounted over a fixed-length episode, so d is only meaningful at the episode length it was calibrated against.

3.5 The grasping task

The task runs on the platform of Section 3.1, is trained with the `rsl_rl` 3.0.1 library [**?**], and is registered as `Isaac-Lift-Cube-UR5e-v0`. It is retargeted from Isaac Lab's Franka cube-lift environment onto the UR5e of Section 3.2, equipped with a Robotiq 2F-85 gripper. Only the robot articulation, the action space and the end-effector frame are replaced, so the reward shaping and the task structure of a well-tested base environment are preserved rather than rewritten.

The arm is controlled by joint-position commands, the gripper by a single binary open/close command on the Robotiq actuation joint. The object is a rigid cube (Isaac's DexCube asset scaled to 0.8) initialised on a table in front of the arm, and its target is a goal pose sampled once per episode.

Contact-based grasping does not hold the cube in this asset. A gripper diagnostic traced the cause: in the converted USD file every gripper rigid body is reported at the wrist origin rather than distributed along the gripper body, while the pad-to-pad separation is reproduced correctly. The linkage is therefore intact but its transform relative to the wrist is degenerate, so finger contact cannot resolve where the geometry is drawn, which accounts for the cube falling through the fingers regardless of clamp force.

Because the Layer 1 result, the safe-RL comparison delivered by this thesis, is independent of the grasp mechanism, the gripper is modelled as a lumped mass at the wrist flange and contact grasping is replaced by a *proximity-weld* abstraction. The tool centre point is defined kinematically as a fixed 160 mm offset along the `wrist_3_link` approach axis (the position a correctly mounted 2F-85 pad midpoint would occupy), and when the policy commands a close action while the cube lies within a 6 cm tolerance of that frame, the cube latches and its pose tracks the end-effector each control step (with its velocity zeroed); an open command releases it and physics resumes. This makes "grasp-and-lift" reliable so that both the baseline and the constrained policy can learn the task, and defers realistic finger contact to the Layer 3 sim-to-real work.

Two consequences follow. The lumped-mass gripper alters the wrist inertia relative to a fully articulated model, but the same asset is used for every run, so this affects all arms identically and cannot confound the comparison. And with self-collision disabled and the collision term reduced to a table-plane keep-out, learned configurations are not guaranteed to be self-collision-free on hardware. Both are revisited in the Limitations discussion.

3.5.1 State, action, and reward

Table 3.5 gives the observation, the action and the episode structure.

Table 3.5. Observation, action and episode.

Element	Definition
Observation	Arm joint positions and velocities (relative to defaults), cube position in the base frame, commanded goal position, previous action. All terms clamped to a finite range
Action	6 target arm-joint positions (scale 0.5) + 1 binary gripper command
Episode	5.0 s = 250 control steps at 50 Hz, decimation 2 physics steps per action
Goal sampling	Once per episode, uniform over x [0.22, 0.60], y [−0.30, 0.30], z [0.10, 0.50] m
Cube reset	Randomised within a small planar region

Two of those rows carry a decision rather than a value. The object pose is given to the policy directly instead of being estimated from an image, which isolates the safe-RL contribution from perception; closing that loop with vision was Layer 2 of the original programme, not attempted here. And the goal box is bounded by reachability, not convenience, as Figure 3.3 shows. Clamping the observations is a numerical safeguard that stops one transiently unstable environment from corrupting the whole on-policy batch.

The reward is the shaped sum inherited from the base lift task, with weights retuned for this

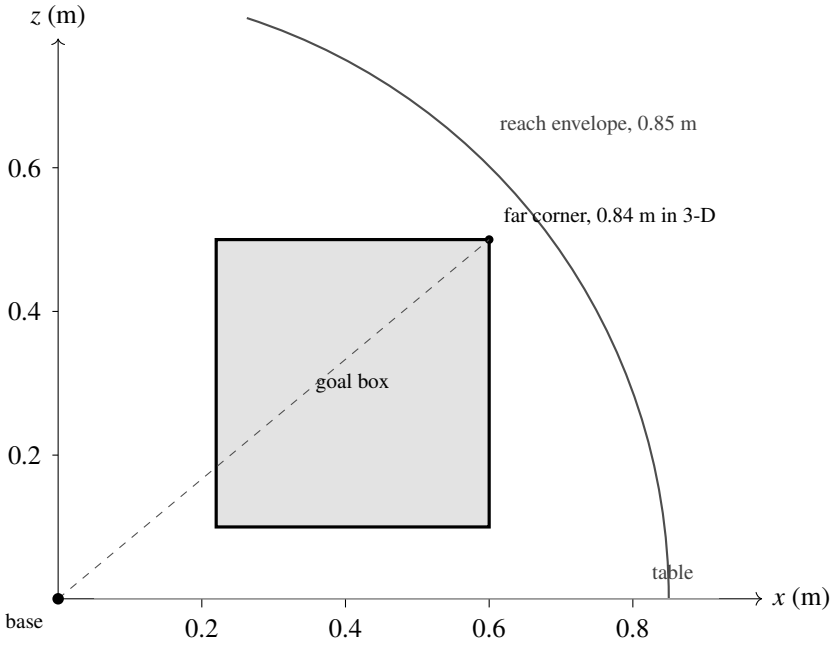

Figure 3.3. The goal-sampling box against the arm's reach envelope, seen in the vertical plane through the base. The far corner of the three-dimensional box, which includes the $y = \pm 0.30$ m extent not visible in this section, lies 0.84 m from the base against a rated reach of 0.85 m.

goal box (Table 3.6). All three lift-dependent terms gate on the cube having climbed half of the distance from the table to the goal height sampled for that episode, rather than on a fixed absolute height, because a fixed threshold scales badly once the box spans 0.10 m to 0.50 m in z. Shaped distance-and-lift rewards of this form, tuned per task rather than derived, are the standard construction in learned contact-rich manipulation [**?**], so the design is conventional and only its safety treatment is not.

Table 3.6. Reward terms.

Term	Form	Weight
Reaching	tanh kernel on end-effector-to-object distance, $\sigma = 0.1$	1
Lifting	Gated on 50 % of the climb to the goal height	10
Goal tracking, coarse	tanh kernel on cube-to-goal distance, $\sigma = 0.3$	15
Goal tracking, fine	tanh kernel on cube-to-goal distance, $\sigma = 0.05$	5
Action rate	Quadratic penalty	-10^{-4}
Joint velocity	Quadratic penalty	-10^{-4}
Safety	**None.** Safety is a separate cost channel	**n/a**

The last row is the one that matters for the comparison. The reward contains no safety term

at all, so safety is expressed entirely through the cost channel of Section 3.7 and the Results chapter can attribute any safety difference to the constraint. Table 3.7 collects the scene settings not already given above.

Table 3.7. Scene and simulation settings.

Item	Setting
Simulator	Isaac Sim 5.0.0, PhysX, GPU-resident
Framework	Isaac Lab 2.3.0, manager-based
Task	`Isaac-Lift-Cube-UR5e-v0`, retargeted from the Franka cube-lift task
Robot	UR5e, 6 revolute joints, fixed base at the environment origin
End effector	Robotiq 2F-85, modelled as a lumped mass at the wrist flange
Grasp model	Proximity weld, 160 mm TCP offset, 6 cm latch tolerance
Object	DexCube asset, scale 0.8, reset on the table in front of the arm
Self-collision	Disabled; collision cost is a table-plane keep-out
Parallel environments	4096 (training), 128 (evaluation)

3.6 Software framework

Every piece of code written for this thesis lives in one Python package, `ur5_grasp`, installed beside Isaac Lab rather than inside it. Keeping it outside the simulator's own source tree has two practical consequences. The environment, the safety cost and the constrained algorithm are versioned and frozen as a single unit, which is what makes a run reproducible. It also keeps the boundary visible between what this work contributes and what it inherits from the framework. Table 3.8 lists the modules.

The safety cost is computed by `SafetyCostComputer`. It resolves the arm joint indices, the monitored body indices and the Jacobian body-axis index once on the first call and caches them, and all arithmetic is batched across environments with nothing copied back to the host. At 4096 environments a per-step host synchronisation would cost more than the physics step it is measuring. The class returns the aggregate cost that the constraint acts on, and alongside it a mean value and a binary violation indicator for each of the three terms, so the constraints can be reported separately even though a single multiplier governs them jointly.

The four `safe_rl` modules extend `rsl_rl` [**?**] rather than replacing it with a different library, and two details of how they attach matter later. `ActorCriticCost` builds the cost critic *after* the base class has finished, so the actor and reward critic draw their initial weights

Table 3.8. The ur5_grasp package.

Module	Role
robots/ur5e_robotiq.py	UR5e articulation, actuator gains, joint speed and effort limits
tasks/lift/ur5e_lift_env_cfg.py	Task configuration: action space, goal-sampling box, reward terms
tasks/lift/ur5e_lift_env.py	Environment class, safety thresholds, per-step cost and safety/* logging
tasks/lift/rewards.py	Goal-relative lift gate used by the three lift-dependent reward terms
safe_rl/costs.py	SafetyCostComputer: the three cost terms of Section 3.7
safe_rl/actor_critic_cost.py	Actor, reward critic and cost critic
safe_rl/rollout_storage_cost.py	Rollout buffer carrying the cost stream and cost advantages
safe_rl/ppo_lagrangian.py	Combined advantage, dual update, partitioned gradient clip
safe_rl/lagrangian_runner.py	Runner that assembles the constrained agent
tasks/lift/agents/	Per-arm hyperparameter configurations (baseline, control, cPPO)
scripts/train.py	Training entry point for every arm
scripts/eval_policy.py	Evaluation of a frozen checkpoint (Section 3.11)
tools/	Asset builders, gripper diagnostics, run summarisers, regression tests

from the same random state as the unconstrained baseline's. LagrangianRunner replaces exactly one method of the stock runner, _construct_algorithm, which is necessary only because the stock runner resolves class names inside its own module namespace, where classes defined outside the library are invisible. Everything else, the rollout loop, the logging and the checkpointing, is inherited unchanged.

Both entry points are single scripts, so the arms cannot diverge by accident: the baseline and the constrained agents are launched by the same command with one configuration flag changed. Each run also writes its resolved environment and agent configuration to disk beside its checkpoints, so two runs are compared by differencing their dumped configurations rather than by trusting that the same flags were passed.

One implementation detail of that extension has to be stated here because the experimental design depends on it. Stock PPO applies a single gradient-norm clip across every parameter handed to the optimiser. In a constrained agent that parameter list also holds the cost critic, so the cost critic's gradients enter the norm and shrink the actor's step even when the multiplier is zero, which makes the constrained agent an optimiser change rather than purely

a constraint. The clip is therefore partitioned, the actor and reward critic in one group and the cost critic in another, so the baseline half of the network is treated exactly as the unconstrained agent's is. Because a correction of this kind cannot be verified by reading the code, a control arm was added that carries the cost critic with its multiplier ceiling pinned to zero. Section 4.2 reports that verification and the withdrawn result that prompted it.

Key points

- All contributed code sits in one package outside the simulator tree, so the environment, the cost and the algorithm are frozen together.
- The constrained agent is a thin extension of the same trainer the baseline uses, which is what lets a measured difference be attributed to the constraint rather than to two different implementations of PPO.
- The gradient clip is partitioned so that the cost critic cannot alter the actor's step, and a control arm exists to verify that it does not.

3.7 The safety cost function

Safety is encoded by a per-step cost, computed by the cost class of Section 3.6 from three geometric-kinematic terms. Each term is zero while the arm is safe and grows smoothly as the arm enters a danger zone; smoothness matters because it gives the cost critic (Section 3.8) a usable gradient. The three terms are summed into a single aggregate cost so that a single Lagrange multiplier governs the whole constraint, and each term additionally exposes a mean value and a binary violation indicator for reporting.

Table 3.9 defines the three terms.

Table 3.9. The three safety cost terms. Each is summed over the bodies or joints it monitors, and the three are added with unit weights.

Term	Per-step penalty	Threshold	Status
Collision keep-out	max(z_floor − z, 0), summed over forearm, wrist-1 and wrist-3	z_floor = 0.05 m	Monitored, inactive
Joint-limit margin	clamp(1 − clearance / margin, 0, 1), summed over the 6 arm joints	margin = 0.175 rad (10.0°)	**Active**
Singularity floor	clamp(1 − w / w_floor, 0, 1) on the manipulability measure w	w_floor = 0.06	**Active**

The third term is the operative constraint of this thesis. The Yoshikawa manipulability measure $w = \sqrt{\det(JJ^\top)}$ [**?**] is computed from the 6×6 end-effector Jacobian J of the arm. A small w means a near-singular configuration, one in which the arm momentarily loses the ability to move in some Cartesian direction and a modest Cartesian command produces large,

jerky joint velocities. The Jacobian body-axis index is resolved automatically to accommodate the fixed-base articulation, for which PhysX omits the root body from the Jacobian, and correctness was confirmed at calibration by a smooth, small-positive distribution of w.

Figure 3.4 shows the shape of that penalty against the baseline's own distribution of w. The ramp is what makes the term learnable: a binary violation flag would be zero almost everywhere and would give the cost critic no gradient to descend, whereas this form tells the agent how close it is to trouble as well as whether it is in it.

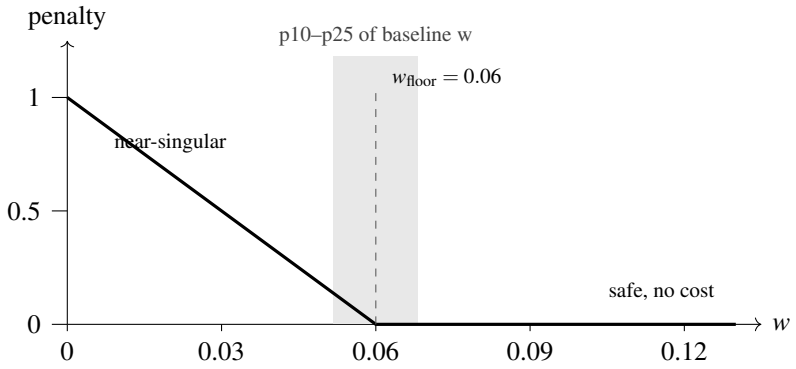

Figure 3.4. The singularity penalty against the manipulability measure w. The floor sits at 0.06, inside the tenth-to-twenty-fifth percentile band of the unconstrained baseline's own distribution, so the baseline pays a cost on roughly a fifth of its steps.

Two details of the other terms are worth stating. The collision floor is a 5 cm standoff above the table rather than the table surface itself, so the term begins to cost before a link penetrates anything; with self-collisions disabled and no contact sensors present, this geometric proxy is the honest and inexpensive choice. The joint-limit penalty has the same ramp shape, zero until a joint enters the final margin band and rising linearly to one at the limit.

The aggregate per-step cost is c = c_collision + c_joint + c_singularity, and its undiscounted sum over an episode is the quantity constrained against the budget d.

Key points

- Three hazards are scored: proximity to the table, proximity to a joint limit, and proximity to a kinematic singularity.
- Every term is smooth rather than a switch, because the cost critic needs a gradient to learn from. A binary violation flag would give it none.
- The singularity term uses the Yoshikawa manipulability measure and is the constraint this thesis is built around; the joint-limit term turned out to be active as well, which Section 3.9 explains.
- The three are summed into one number before the constraint sees it, so one multiplier governs all three, but each is logged separately for reporting.

3.8 Policy optimisation: PPO and cPPO

This section develops the algorithm that solves the CMDP of Section 3.4. It is built in two stages, because the constrained method changes exactly one term of the unconstrained one and the change is far easier to follow once the unmodified objective is on the page. Proximal Policy Optimisation, PPO, is the unconstrained baseline [**?**]; the constrained arm, written cPPO throughout, is its Lagrangian variant.

A policy-gradient method improves a parameterised policy π_θ by ascending an estimate of the gradient of expected return, in which each action is weighted by its advantage $\hat{A}_t$, that is, by how much better it was than the policy's average behaviour in that state. Ascending that estimate directly is unreliable, because it is only valid near the policy that collected the data: a step large enough to be useful is often large enough to move the policy somewhere the estimate no longer describes. Trust-region methods bound the policy change explicitly, at the cost of a second-order optimisation. PPO reaches the same end with a first-order method by building the bound into the objective.

3.8.1 The clipped surrogate objective

Let $\pi_{\theta_{\text{old}}}$ be the policy that collected the current batch. PPO works with the probability ratio between the candidate policy and that one,

$$r_t(\theta) \;=\; \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad r_t(\theta_{\text{old}}) = 1. \tag{3.2}$$

The ratio measures how much more, or less, likely the candidate policy is to have taken the action that was actually taken. An unconstrained surrogate would maximise $r_t(\theta)\hat{A}_t$, which grows without bound as the policy moves further from the one that generated the data. PPO instead maximises

$$L^{\text{CLIP}}(\theta) \;=\; \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\; \text{clip}\big(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon\big)\hat{A}_t\Big)\Big], \tag{3.3}$$

with clipping range $\varepsilon = 0.2$ in this work. The behaviour of Equation 3.3 is asymmetric in a way worth stating plainly. Where the advantage is positive, the action was better than average and the objective would reward pushing r_t up, but the clip caps the gain at $1+\varepsilon$, so there is no incentive to move the policy further than that. Where the advantage is negative, the same cap applies at $1-\varepsilon$ in the other direction. The minimum of the clipped and unclipped terms makes the objective a pessimistic bound on the unclipped one, so the update never benefits from a large policy change, only from a well-directed small one. A single scalar therefore does the work a trust region does, without a constrained optimisation.

3.8.2 *Advantage estimation and the full objective*

The advantage itself is estimated by generalised advantage estimation over a truncated rollout of length T. Writing the temporal-difference residual as

$$\delta_t \;=\; r_t + \gamma V(s_{t+1}) - V(s_t), \tag{3.4}$$

the estimate is the exponentially weighted sum

$$\hat{A}_t \;=\; \sum_{l=0}^{T-t-1} (\gamma\lambda_{\mathrm{GAE}})^l \, \delta_{t+l}, \tag{3.5}$$

in which γ discounts future reward and λ_{GAE} sets where the estimate sits between a one-step bootstrap, at $\lambda_{\mathrm{GAE}} = 0$, which has low variance and high bias, and a full Monte-Carlo return, at $\lambda_{\mathrm{GAE}} = 1$, which has the opposite. This work uses $\gamma = 0.98$ and $\lambda_{\mathrm{GAE}} = 0.95$. The subscript is carried deliberately: from Section 3.8.3 onward an unsubscripted λ always means the Lagrange multiplier, which is a different quantity entirely.

Because the value function V appearing in Equation 3.4 is itself learned, and because a policy that collapses to a single action stops exploring, the quantity actually optimised combines three terms:

$$L(\theta) \;=\; \hat{\mathbb{E}}_t\Big[L_t^{\mathrm{CLIP}}(\theta) \;-\; c_1\left(V_\theta(s_t) - V_t^{\mathrm{targ}}\right)^2 \;+\; c_2\, S\big[\pi_\theta\big](s_t)\Big], \tag{3.6}$$

where the second term fits the value network to its bootstrapped target with coefficient $c_1 = 1.0$, and the third is an entropy bonus with coefficient $c_2 = 0.006$ that keeps the policy stochastic for long enough to explore. Each batch of experience is then used for several epochs of minibatch gradient steps, five epochs of four minibatches here, which recovers part of the sample efficiency an on-policy method gives up by discarding its data. The implementation used in this work additionally adapts the learning rate between iterations to hold the observed Kullback-Leibler divergence between successive policies near a target of 0.01, which is a second, softer guard on the same failure the clip addresses.

3.8.3 *The constrained variant*

cPPO changes Equation 3.6 in one place and leaves the rest alone. Figure 3.5 shows the whole agent, with everything cPPO adds over the baseline drawn inside the dashed region: a second critic, a scalar Lagrange multiplier, and the dual-ascent loop that sets the multiplier from measured violation rather than from a designer's guess. The method is PPO-Lagrangian as specified and benchmarked by Ray et al. [**?**], within the constrained policy optimisation

family founded by Achiam et al. [**?**]. Their benchmark is also the reason the Lagrangian variant is preferred here over CPO itself, which it reports as the weaker of the two on the same tasks.

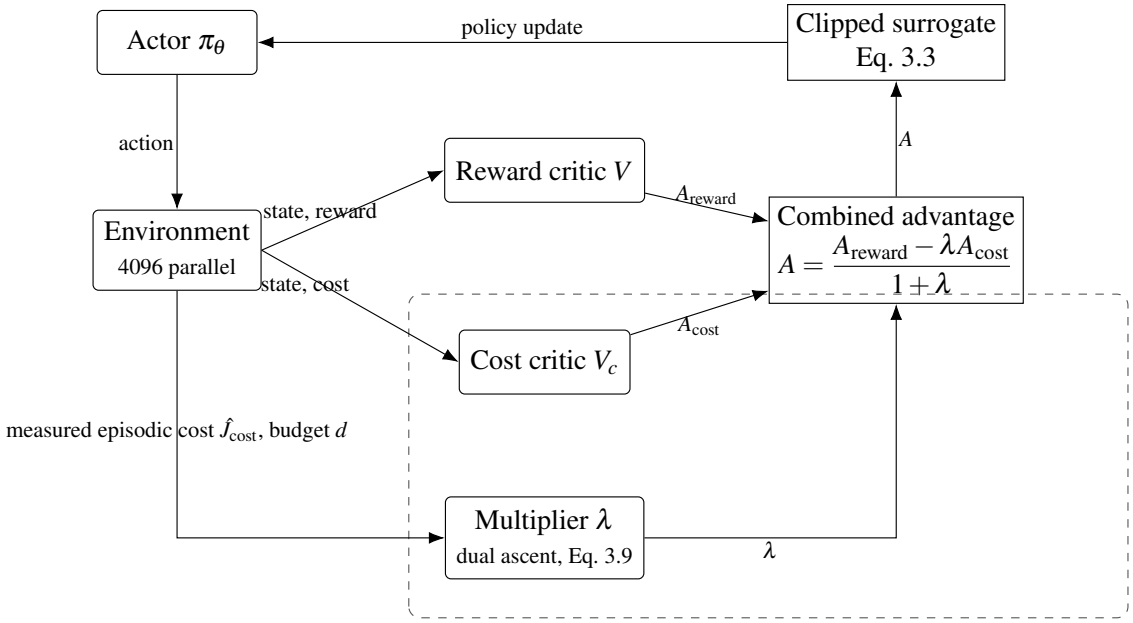

Figure 3.5. The cPPO agent. The dashed region marks what the constrained arm adds to the baseline: a cost critic and a multiplier updated by dual ascent on measured violation. Their only point of entry into the policy update is the combined advantage, and pinning λ to zero recovers the baseline exactly.

The method converts the constrained problem into the saddle-point objective

$$\max_{\pi} \min_{\lambda \geq 0} \quad J_{\text{reward}}(\pi) - \lambda \left(J_{\text{cost}}(\pi) - d \right), \tag{3.7}$$

where λ is a non-negative Lagrange multiplier acting as an automatically-tuned penalty on constraint violation. The implementation, whose modules are listed in Table 3.8, follows the textbook separate-critic variant: in addition to the usual value network that estimates future reward, a second critic estimates future cost, so reward and cost advantages are learned independently. Cost advantages are computed by generalised advantage estimation with its own discount and smoothing (γ_cost = 0.98, λ_GAE = 0.95).

At each policy-update step the reward and cost advantages are combined into a single Lagrangian advantage,

$$A = \frac{A_{\text{reward}} - \lambda A_{\text{cost}}}{1 + \lambda}, \tag{3.8}$$

which is then used in the standard clipped PPO surrogate; the division by (1 + λ) keeps the effective step size stable as λ grows. The multiplier itself is updated once per iteration by projected dual ascent on the measured mean episodic cost of the just-collected rollout,

$$\lambda \leftarrow \mathrm{clip}\big(\lambda + \eta\,(\hat{J}_{\mathrm{cost}} - d),\ 0,\ \lambda_{\max}\big), \tag{3.9}$$

with dual step η = 0.035, initial λ = 0, and ceiling λ_max = 100. Intuitively, whenever the policy exceeds its cost budget the multiplier rises and unsafe actions are penalised more heavily; once the policy is comfortably under budget the multiplier decays back toward zero. The multiplier tunes itself during training and is never hand-set.

Setting $\lambda = 0$ in Equation 3.8 returns $A = A_{\mathrm{reward}}$, so the unconstrained baseline is this same algorithm with the multiplier pinned at zero. Building the constrained agent as a thin extension of the baseline, rather than adopting a separate safe-RL library, is a deliberate methodological choice: it guarantees that the two share an identical trainer, network architecture and hyperparameters, so any measured difference is attributable to the safety constraint alone and not to two dissimilar PPO implementations.

Both agents use the same actor and critic multilayer perceptrons with hidden layers of 256, 128 and 64 units and ELU activations, and the same optimisation hyperparameters, summarised in Table 3.10.

Table 3.10. Shared training hyperparameters.

Hyperparameter	Value
Parallel environments	4096
Rollout length (steps/env)	24
Iterations	1500
Actor / critic hidden layers	[256, 128, 64], ELU
Clip parameter	0.2
Entropy coefficient	0.006
Learning epochs / mini-batches	5 / 4
Learning rate (adaptive, target KL 0.01)	1×10^{-4}
Discount γ / λ_{GAE}	0.98 / 0.95
Max gradient norm (per group)	1.0
cPPO cost budget d (`cost_limit`)	25, and 15 for the tighter arm
cPPO dual step η / λ ceiling	0.035 / 100
cPPO cost discount / λ_{GAE}	0.98 / 0.95

Key points

- PPO's central idea is a probability ratio between the new policy and the one that collected the data, clipped so that a large policy change can never be rewarded.
- The clipped objective is a pessimistic bound on the unclipped one, which is how a first-order method achieves what a trust region does second-order.

- Advantages come from generalised advantage estimation, where λ_{GAE} trades bias against variance. It is not the Lagrange multiplier, which shares the letter.
- cPPO changes exactly one quantity in that objective, the advantage. The clip, the value loss, the entropy bonus and the epoch structure are identical to the baseline, and the baseline is this algorithm with the multiplier pinned at zero.
- The multiplier is set by dual ascent on measured violation, so the safety-versus-task weight is never hand-tuned. This is the property a shaped reward cannot offer.

3.9 Constraint calibration and cost budget

A safety benchmark is only meaningful if the unconstrained baseline actually violates the constraint; otherwise the constrained policy has nothing to demonstrate. All three thresholds and the budget were therefore measured from a converged 1500-iteration unconstrained agent, run against the final environment immediately before the code was frozen. Calibrating against a converged policy rather than a short probe matters here, because a policy fifty iterations into training has barely begun to reach and its cost distribution says little about the one that would be deployed. Table 3.11 gives what was measured and what each measurement settled.

Table 3.11. Calibration against the converged unconstrained baseline.

Quantity	Measured on the baseline	Outcome
Manipulability w	min 0.0171, mean 0.0750, max 0.1109; p10 = 0.0517, p25 = 0.0682	Floor set to 0.06, about p18, so the baseline violates on roughly a fifth of steps
Joint-limit clearance	33.7 % of steps inside the margin; minimum clearance reaches the soft limit	Margin 0.175 rad is **active**, and is the larger of the two active terms
Monitored-link height	Minimum 0.0896 m, about 44 mm above the floor	Collision floor 0.05 m stays inactive
Cost composition	Joint-limit $\approx$ 86 %, singularity $\approx$ 14 %, collision 0 %	Two constraints share one budget
Natural episodic cost	$\approx$ 105	Budget d = 25, a $\approx$ 76 % reduction

Three consequences follow. The manipulability floor was raised to 0.06 from an earlier value of 0.045, which had been set against a narrower goal-sampling box and no longer produced a violation rate inside the intended band once the workspace was widened. Joint-limit proximity is no longer the monitored-but-satisfied term it was in earlier drafts, so two active constraints now share one budget and one multiplier, the agent is free to trade between them, and Chapter 4 reports each separately for that reason. And the budget is an undiscounted sum over a per-step cost across a fixed 250-step episode, so it is tied to the five-second episode

length: changing that length would rescale the constraint and void the calibration.

3.10 Training protocol

Table 3.12 states the protocol as run. The one thing it cannot state is what a successful run looks like while it is in progress. The healthy Lagrangian signature to be verified during training is that the mean episodic cost trends down toward the budget, the multiplier rises while over budget and then settles without pinning at its ceiling, the reward continues to improve, and the singularity-violation rate falls below the baseline.

Table 3.12. Training protocol.

Item	Setting
Arms	3: PPO (baseline), cPPO at d = 25, cPPO at d = 15
Seeds per arm	5: 1, 3, 4, 52, 54
Trained policies	15
Iterations	1500 per run
Parallel environments	4096
Rollout length	24 steps per environment per iteration
Throughput	$\approx 2 \times 10^5$ environment steps/s on an RTX 5090
Library	`rsl_rl` 3.0.1, PyTorch 2.7, CUDA 12.8
Execution	Headless, monitored through TensorBoard
Launch	One script, arm selected by agent-config entry point
Logging	Separate experiment directory per arm, so curves overlay directly
Verification	All 15 final checkpoints confirmed present on disk, not inferred from clean logs

3.10.1 Experimental arms and seeds

Three arms were trained, listed in Table 3.13. They differ only in the two fields shown; network architecture, hyperparameters, environment and trainer are identical across all three. The second constrained budget is a sensitivity check on the calibration rather than a separate method: 15 binds more deeply than 25 on the seeds where either binds at all, so the pair shows whether the effect of the constraint scales with how hard it is applied.

Table 3.13. The three experimental arms.

Arm	`cost_limit`	$\lambda_{\max}$	Reported in this thesis as
`ctrl`	25	0	PPO (baseline)[1]
`cppo`	25	100	cPPO, d = 25
`cppo15`	15	100	cPPO, d = 15

Each arm was trained on five random seeds, 1, 3, 4, 52 and 54, giving fifteen trained policies. Reporting five seeds and their dispersion, rather than a single run or a mean alone, follows the reproducibility argument made by Henderson et al. [**?**], who show that deep policy-gradient results on continuous control can differ qualitatively between seeds of the same algorithm and that a mean over too few runs can describe a policy none of them produced. Dispersion across seeds is therefore reported as a result in its own right in Chapter 4, not as an error bar attached to one.

3.11 Evaluation protocol

Every reported number is measured after training on the frozen policy, not read from training-time telemetry. This distinction is a correction rather than a preference. An earlier version of this work reported safety percentages taken from the TensorBoard scalars of a policy that was still exploring and still learning, which describes neither the policy that would be deployed nor any policy at all. Each checkpoint is therefore replayed by `eval_policy.py` with the actor set to its deterministic mean action, exploration noise off and observation corruption disabled.

Table 3.14 states the protocol. Three of its choices carry an argument that the rows cannot.

Evaluation seeds are disjoint from training seeds so that variation caused by the exam is separable from variation caused by which policy was learned, and the two turn out to differ by more than an order of magnitude. Goal-reach is reported with the full distribution of final goal distances rather than on its own, because the weld writes the cube's pose to the tool frame at every step, so the 1 cm test reduces to whether the tool centre point reached the goal: a converged policy passes it on almost every episode and an unconverged one fails on almost every episode, which makes a single threshold saturate and hide exactly the differences a distribution shows. And safety is counted per episode rather than averaged per step, so one dangerous excursion stays visible instead of being diluted across 250 steps. The resulting comparison is presented in Chapter 4.

[1] The control arm carries the cost critic with its multiplier ceiling pinned to zero, so its policy update is stock PPO. A plain PPO arm was trained on the same seeds and its stored network weights proved byte-identical to the control arm's, so the control arm stands in for it and the plain arm is not reported separately. The verification is given in Section 4.2.

Table 3.14. Evaluation protocol.

Item	Setting
Policy	Frozen checkpoint, deterministic mean action, exploration noise off
Observation corruption	Disabled
Evaluation seeds	101, 102, 103, disjoint from the training seeds
Episodes	1000 per checkpoint per evaluation seed
Parallel environments	128
Episodes per policy	3000
Episodes per arm	15,000
Lift success	Cube past 50 % of the climb to that episode's goal height
Goal-reach success	Lifted cube within 1 cm of the commanded goal pose
Reported alongside	Full distribution of final goal distances, since the weld makes a single threshold saturate
Safety, per episode	Episodic cost; singularity crossings ($w < 10^{-4}$); episodes touching the joint-limit margin
Constrained arms only	Terminal multiplier, episodic-cost trajectory